

UNIVERSAL SHIFT REGISTER REPORT

Digital Design (25 points)	Verilog Implementation (25 points)	Verilog Testbench (25 points)	Academic Report and Presentation (25 points)	Total (100 points)

	Student Name and Surname	Student ID	Signature
1	Hatice Özates	22040102033	
2	Zkra Osman	22040102088	

<https://github.com/haticeozates/Universal-Shift-Register>

<https://github.com/zkraosmn/Universal-Shift-Register>

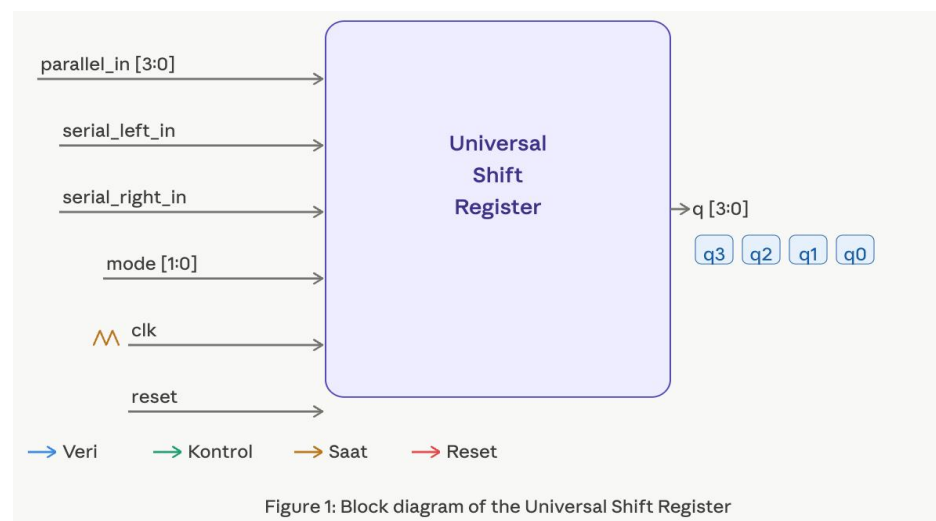
1. Introduction

This project implements a 4-bit Universal Shift Register using Verilog HDL. A universal shift register is a sequential circuit capable of performing multiple operations such as holding data, shifting left, shifting right, and parallel loading.

The main objective of this project is to design, implement, and simulate a shift register that supports all these operations based on a mode selection input.

2. Design

The designed register supports four different operations depending on the mode input:



This diagram shows how different inputs control the behavior of the universal shift register.

00 → Hold (No change)

01 → Shift Right

10 → Shift Left

11 → Parallel Load

Inputs:

- clk: Clock signal
- reset: Resets the register
- mode[1:0]: Operation selector
- parallel_in[3:0]: Data input for parallel load
- serial_left_in: Input for shift left
- serial_right_in: Input for shift right

Output:

- q[3:0]: Register output

The register updates its value on the positive edge of the clock signal.

3. Verilog Implementation

The following Verilog code was used:

Listing 1: Verilog implementation of the universal shift register

```
module universal_shift_register (
    input wire clk,
    input wire reset,
    input wire [1:0] mode,
    input wire [3:0] parallel_in,
    input wire serial_left_in,
    input wire serial_right_in,
    output reg [3:0] q
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        q <= 4'b0000;
    end
    else begin
        case (mode)
            2'b00: q <= q;
            2'b01: q <= {serial_right_in, q[3:1]};
            2'b10: q <= {q[2:0], serial_left_in};
            2'b11: q <= parallel_in;
        endcase
    end
end

endmodule
```

4. Verilog Testbench

```
`timescale 1ns/1ps

module tb_universal_shift_register;

    reg clk;

    reg reset;

    reg [1:0] mode;

    reg [3:0] parallel_in;

    reg serial_left_in;

    reg serial_right_in;

    wire [3:0] q;

    universal_shift_register uut (

        .clk(clk),

        .reset(reset),

        .mode(mode),

        .parallel_in(parallel_in),

        .serial_left_in(serial_left_in),

        .serial_right_in(serial_right_in),

        .q(q)

    );

    initial begin

        clk = 0;

        forever #5 clk = ~clk;

    end

    initial begin

        $dumpfile("universal_shift_register.vcd");

        $dumpvars(0, tb_universal_shift_register);

        reset = 1;

        mode = 2'b00;
```

```
parallel_in = 4'b0000;

serial_left_in = 0;

serial_right_in = 0;

#10 reset = 0;

// Parallel Load: q = 1011

#10 mode = 2'b11; parallel_in = 4'b1011;

// Hold: q stays 1011

#10 mode = 2'b00;

// Shift Right: serial_right_in enters from left

// q = 0101

#10 mode = 2'b01; serial_right_in = 0;

// Shift Left: serial_left_in enters from right

// q = 1011

#10 mode = 2'b10; serial_left_in = 1;

// Parallel Load: q = 1100

#10 mode = 2'b11; parallel_in = 4'b1100;

// Hold: q stays 1100

#10 mode = 2'b00;

#20 $finish;

end

endmodule
```

5. Simulation and Results

The design was verified using a testbench.

A waveform was generated using a VCD file and visualized.

The output changes only at clock edges, confirming correct sequential behavior.

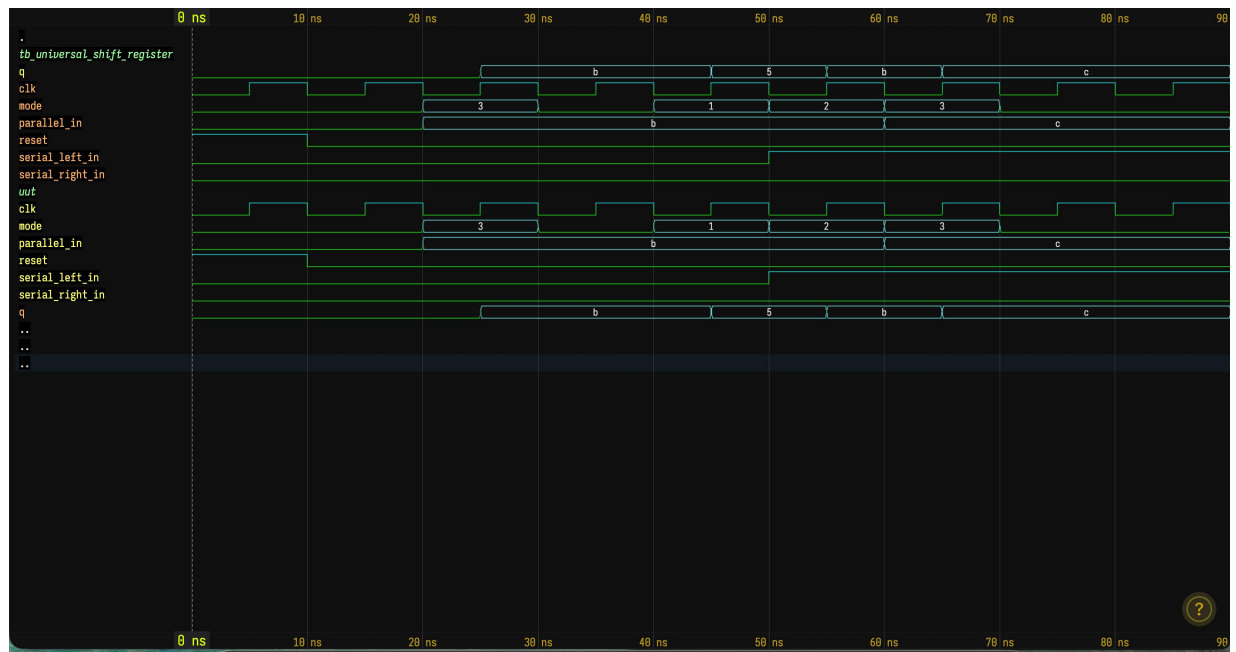


Figure 2: Simulation waveform of the universal shift register

Explanation:

At 20 ns, the mode selector becomes '11' and the parallel input 'b' (1011) is successfully loaded. At 40 ns, the mode switches to '01' (Shift Right), and with `serial_right_in=0`, the output shifts to '5' (0101). Finally, at 50 ns, a left shift operation (mode '10') with `serial_left_in=1` is observed, restoring the value back to 'b'.

The waveform confirms that all operations are working correctly.

How to Run:

The design can be compiled and simulated using Icarus Verilog.

Step 1: Compile the design and testbench

```
iverilog -o simulation.out universal_shift_register.v tb_universal_shift_register.v
```

Step 2: Run the simulation

```
vvp simulation.out
```

After running the simulation, a VCD file (universal_shift_register.vcd) is generated, which can be viewed using waveform viewers such as GTKWave.

6. Conclusion

In this project, a Universal Shift Register was successfully designed and implemented using Verilog.

All required operations were tested and verified through simulation.

The results demonstrate that the circuit behaves correctly according to the given mode input.

This project helped to understand sequential circuits and shift register operations.